

SegmaAI

Customer Segmentation & Personalization Agent for Retail
Banking

PROJECT DOCUMENTATION



Prepared by

Sai Aravind.S & Rahul Vinod

What's in this document

- 01 **Executive Summary** What SegmaAI is, in three sentences

- 02 **The Problem** Why one-size-fits-all banking falls short

- 03 **Solution Overview** Six capabilities behind one conversation

- 04 **System Architecture** How a query becomes an answer

- 05 **Data & Provenance** What's real, what's calibrated, what's synthetic

- 06 **Reliability Engineering** Autonomy, fallbacks, and a real test suite

- 07 **Technology Stack** What runs where

- 08 **Using SegmaAI** Example queries & how to run it

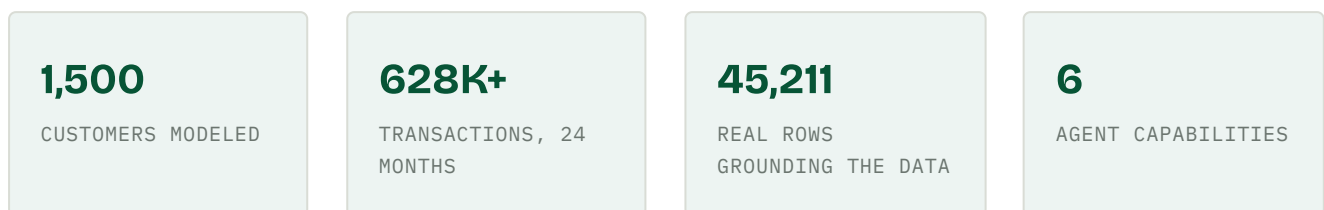
- 09 **Team & Closing Notes** Who built it, what's next

How to read this document. Each section stands on its own — skip straight to Architecture (04) or Data (05) if that's what you're here for. Code references like `backend/agent/planner.py` point at the actual source file for anyone who wants to go deeper than this summary.

An agent that segments customers and explains itself

SegmaAI is a conversational agent for retail banking that takes a plain-English question — "segment customers by balance and transaction frequency," "which regular customers can become priority?" — and turns it into a real answer, computed by actual statistics and machine learning, not guessed by a language model.

The core design decision runs through everything else in this document: **the language model plans and explains, but it never calculates.** A query is first translated into a structured instruction (an "intent"), that instruction is executed by deterministic pandas and scikit-learn code, and only the final, already-correct answer is handed back to a language model to phrase in plain English. This keeps every number reproducible, auditable, and fast — and means the whole system still works with zero AI API keys configured, falling back to a rule-based planner and templated explanations.



02 — The Problem

A retail bank sits on rich customer data — balances, transaction history, product holdings, credit behavior — but typically markets to everyone the same way. A high-balance, high-engagement customer and a barely-active one get the same email. The bank wants to segment its customers into meaningful groups, understand *why* each customer belongs where they do, and act on that understanding — recommend the right product, flag who's about to churn, identify who's close to becoming a top-tier customer.

Doing this well requires three things most one-off analytics scripts don't provide together: a natural-language interface a non-technical stakeholder can actually use, segmentation logic that can be explained and defended (not a black box), and a data foundation rich enough to answer follow-up questions about behavior over time — not just a single demographic snapshot.

Six capabilities, one conversation

Segmentation — two ways to slice the customer base

Rule-based tiering sorts customers into **Priority / Regular / Dormant** using an explicit, auditable threshold on balance *and* transaction frequency together. Unsupervised ML clustering (K-Means, with the number of clusters chosen automatically via silhouette score) runs the same job across income, balance, spend and credit behavior when the data should speak for itself instead of following a pre-set rule. A compliance-facing decision needs a threshold a banker can point to; a discovery question needs the data to reveal its own groups — one method can't serve both, so both exist.

Explainability — every segment comes with a reason

Ask "why is this customer here" and get the actual rule thresholds that placed them, or which features most distinguish their cluster from the others — never a label with no justification. A segmentation a banker can't explain to a customer, or a regulator, is a liability, not an insight.

Recommendations & conversion candidates

Per-tier cross-sell and retention actions, plus a dedicated scoring pass that finds which Regular customers sit closest to Priority-tier thresholds — who converts with the least push. A tier label is descriptive; the business value is in knowing what to *do* about each one.

EDA dashboard

Missing-value checks, distribution histograms, correlation analysis, and dataset-wide overview stats — in natural language or from a dedicated dashboard page. Trusting a segmentation means first understanding the data underneath it.

Customer lookup

Pull any customer's full profile and, once segmented, a plain-language explanation of why they landed in their tier. Segment-level answers serve strategy; a relationship manager's day-to-day questions are about one customer at a time.

Trend & time-based queries

"Find customers whose balance has steadily increased" surfaces behavioral trends — balance trajectory, six-month spend change — that a static tier label can't show. Often the more actionable signal of the two.

How a query becomes an answer

Two pipelines meet in the middle: a **data pipeline** that turns three raw tables into one analysis-ready view, and a **request pipeline** that turns a question into that answer. The language model only appears at the two ends of the request pipeline — understanding the question, and phrasing the response.

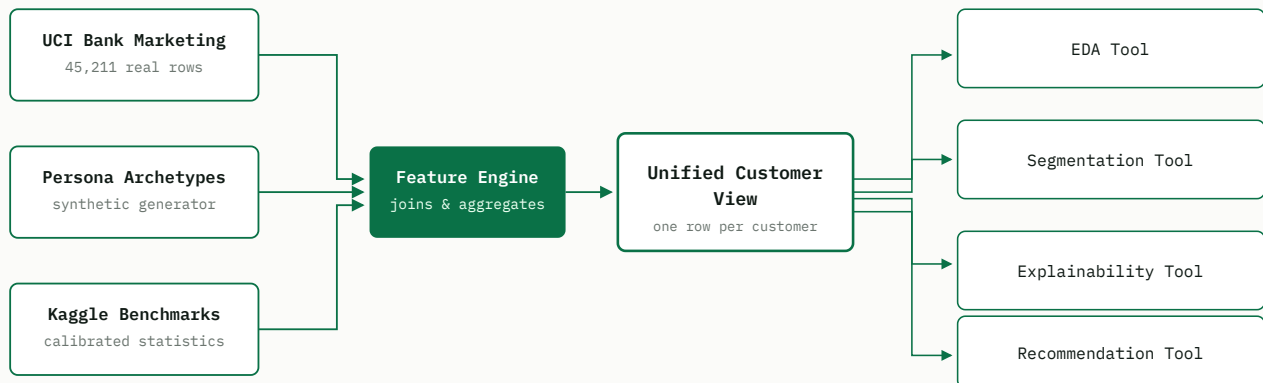


Fig. 1 – Data pipeline: three sources become one analysis-ready view, feeding four independent tools

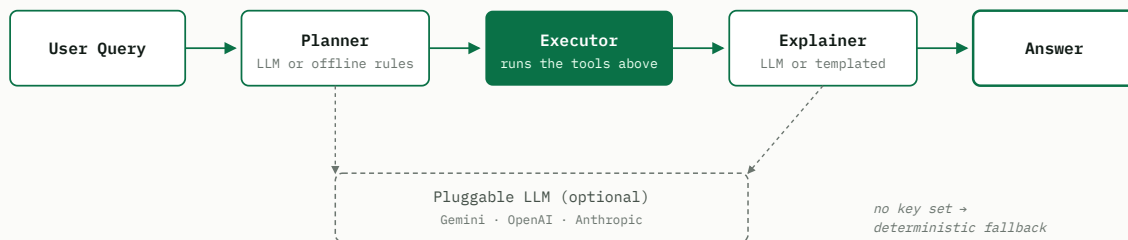


Fig. 2 – Request lifecycle: the LLM plans and explains; the executor's pandas/scikit-learn tools do every calculation

Three design principles

The LLM never touches the numbers

The planner's only job is to turn a sentence into a structured intent — `segment_customers`, `aggregate_metric`, `explain_segment_basis`, `conversion_candidates`, `recommendation`, `list_customers_by_tier`, and a handful of EDA intents. The executor runs that intent through plain pandas and scikit-learn. Only the final human-readable phrasing optionally passes through an LLM. Every figure in every answer is deterministic and reproducible — ask the same question twice, get the same numbers, worded slightly differently.

Works with zero API keys

If no LLM key is configured, the planner falls back to a deterministic keyword/regex parser and the explainer falls back to clear templated text — the entire application remains fully functional offline. Set one key (`GEMINI_API_KEY`, `OPENAI_API_KEY`, or `ANTHROPIC_API_KEY`) to upgrade both stages to a real language model; none of the analytical logic changes either way.

Human-in-the-loop, not silent guessing

An ambiguous request — "segment customers" with no criteria given — returns a clarifying question instead of an arbitrary default. But the bar for "ambiguous" is set carefully: reasonable questions that merely assume a step hasn't happened yet (like asking for a recommendation before any segmentation has run) are handled automatically instead of being bounced back to the user — see Reliability Engineering, next.

Session state, not statelessness. Each conversation keeps its own in-memory `AgentSession` — once a segmentation has run, follow-up questions ("on what basis were priority customers selected?") reuse it instead of recomputing from scratch, the same way a human analyst would keep their working state during a conversation.

A hybrid dataset, honestly labeled

No single public dataset covers demographics, balances, multi-year transaction history, and product ownership together — which is what lets one agent answer EDA, segmentation, explainability, and recommendation questions from the same customer base. So the dataset is a hybrid: real where a real source was usable, calibrated against published statistics where it wasn't, and synthetic everywhere else — every column labeled with which.

FIELD	PROVENANCE	SOURCE / METHOD
Age, marital status, education	real	Bootstrapped from the UCI Bank Marketing dataset, job-matched to each customer's persona
Housing / personal loan uptake	real	Same real dataset — drives the synthetic <code>LoanType</code> assignment directly
Prior campaign response	real	Contact count, recency, and outcome from real historical campaign records
Account balance	real-ranked	Synthetic in scale, but quantile-mapped to each customer's real balance rank within their job group
Card utilization	calibrated	Beta-distribution shape matching a published ~27.5% mean, right-skewed benchmark
Income, city, credit score, EMI, spend, products	synthetic	Generated from seven persona archetypes for internally consistent correlations
Transaction history	synthetic	24 months, archetype-conditioned frequency and amount, realistic merchant/category mix

What wasn't used, and why. A synthetic transaction dataset (IBM TabFormer) was evaluated but its real transaction file proved undownloadable — the source repository's storage quota was exceeded at the time of building this project. Rather than substitute something misleading, the project's own transaction generator stands in for it, clearly labeled synthetic above.

Seven persona archetypes

The synthetic population is drawn from seven weighted personas — Priority Affluent, Mass Salaried Regular, Young Digital, Dormant Inactive, Retiree Conservative, Credit Stressed, and Business Growth — each controlling a consistent joint distribution of income, balance behavior, spending pattern, and product ownership. This is what makes clustering and EDA on the resulting data find real structure instead of noise, while keeping the true labels hidden from the agent so segmentation is genuinely discovered, not memorized.

Making sure it actually works

An agent this size — planner, executor, six tools, two segmentation methods, two LLM modes — breaks in non-obvious ways when anything changes. Three concrete gaps were found and closed, and a real test suite now exists to prove they stay closed.

Gap 1 — the agent used to refuse its own job

Four tools would check whether a segmentation had already run and, if not, reply "run segmentation first" instead of answering. A first question like "what should be done to re-engage dormant customers?" is a completely reasonable opening move — the user shouldn't need to know segmentation is a separate internal step. **Fix:** those tools now transparently run rule-based tiering the moment it's needed, then continue — the user only ever sees their answer.

Gap 2 — the most obvious question fell through the cracks

"Segment customers by balance and frequency" matched; a simpler, more natural "which customers are marked priority?" or "show me the dormant customers" matched nothing and fell back to a generic clarification. **Fix:** a new `list_customers_by_tier` intent recognizes a wide range of natural phrasings and auto-segments the same way Gap 1's fix does.

Gap 3 — no way to know if any of it actually worked

"Does it work" meant manually typing into the chat UI. **Fix:** a pytest suite now covers three levels of the system —

- **Unit** — segmentation math itself: every customer gets exactly one tier, "Priority" requires balance *and* engagement together (not an inclusive-or bug), nothing crashes on a zero-transaction customer.
- **Integration** — real HTTP requests through the FastAPI test client, proving the previously-gated flows and the new tier-listing intent answer immediately on a brand-new session, with no refusals.
- **Planner accuracy** — a ~20-query parametrized battery checking the offline planner lands the correct intent, plus a matching subset against the real LLM classification path.

Living documentation, not a one-time check. A standalone script drives 15–20 realistic queries against a live running server, each on a fresh session, and writes a human-readable report of query → intent → answer. It's re-runnable at any time as a whole-stack sanity check — and it's how the one remaining known gap in natural-language coverage was found and documented, rather than hidden.

What runs where

LAYER	TECHNOLOGY	ROLE
Frontend	Streamlit	Chat interface + EDA / Segments / Customer / Persona dashboards
Backend API	FastAPI	The agent's HTTP surface — <code>/api/chat</code> and supporting endpoints
Planner & Explainer	Gemini / OpenAI / Anthropic (pluggable)	Optional natural-language understanding & phrasing, swappable via one environment variable
Offline fallback	Rule-based Python	Deterministic keyword/regex planner + templated explainer — zero API keys required
Analytics	Pandas, NumPy	Every aggregation, join, and feature computation
Machine learning	Scikit-learn	K-Means clustering, standard scaling, PCA projection, silhouette scoring
Visualization	Plotly	Charts across every dashboard page
Testing	Pytest, FastAPI TestClient	Unit, integration, and planner-accuracy coverage
Data grounding	UCI Bank Marketing dataset	45,211 real rows behind demographic & campaign-history fields

Why this split

Every box in the table above earns its place by doing one job well: Streamlit for a fast, good-looking analytical UI without hand-rolled frontend infrastructure; FastAPI because the agent's logic should be reachable the same way by a UI, a script, or a judge's `curl` command; pandas/scikit-learn because segmentation and statistics should be boring and correct, not reinterpreted by a language model on every call; and a pluggable LLM layer so the choice of provider is a configuration detail, never an architectural one.

Example queries

ASK	DEMONSTRATES
"Segment customers by balance and transaction frequency"	Rule-based tiering
"Segment customers using ML clustering across income and spend"	Unsupervised K-Means path
"On what basis were priority customers selected?"	Explainability
"Which regular customers can convert to priority?"	Conversion scoring
"What should we recommend to dormant customers?"	Retention recommendations
"Show me the dormant customers"	Tier listing, zero setup required
"Has average balance been trending up?"	Time-based trend detection
"Is credit score correlated with income?"	Exploratory data analysis

Running it

`python data/generate_data.py` builds the dataset once. Then `uvicorn backend.main:app --port 8000` starts the API, and `streamlit run frontend/app.py` starts the interface — no LLM key required for either to work end to end.

Built by

Sai Aravind.S

Project development & engineering

Rahul Vinod

Project development & engineering

SegmaAI was built to show that an agent handling a genuinely open-ended, high-stakes domain like retail banking doesn't have to choose between being conversational and being trustworthy. Every number it states, it can also justify — and every claim in this document about what works, works because it's covered by a test that says so.